TECHNICAL ARCHITECTURE DESIGN DOCUMENT

Microservices Architecture Design
CQRS · Event Sourcing · Outbox Pattern

A complete hands-on blueprint for an event-driven e-commerce platform — Catalog, Shopping Cart, Discount & Ordering services — built with Spring Boot, Apache Kafka, PostgreSQL, Docker and Kubernetes.

CQRS | Event Sourcing | Outbox Pattern | Kafka Eventbus | Materialized Views | Docker | Kubernetes | Spring Boot 3

Prepared July 2026 | Version 1.0

Table of Contents

1. Introduction to Microservices
2. Reference Architecture Diagram
3. Microservices Characteristics
4. Microservices Design Principles (CQRS & Event Sourcing)
5. End-to-End Flow & Sequence Diagram
6. Technology Stack
7. Read Database Replica Creation (Projection Pipeline)
8. The Outbox Pattern & Worker Thread Management
9. Core Code Implementation
10. Deployment — Docker & Kubernetes
11. Microservices Testing Strategy
12. Containers & Orchestration
13. Summary Mapping & Benefits

1. Introduction to Microservices

Microservices are an architectural style where an application is composed of small, independent services that communicate over well-defined APIs. Each service focuses on a single business capability, is independently deployable, and can use its own data store.

1.1 Platform Scenario

This design targets an e-commerce platform decomposed into four business-aligned services, fronted by a single API Gateway and connected through a shared Kafka event bus:

Service	Responsibility
Catalog Service	Manages product metadata (name, description, price, category, stock).
Shopping Cart Service	Manages user carts — add/remove items, view cart contents.
Discount Service	Applies promotional rules and calculates discounts.
Ordering Service	Handles order placement, payment, and order status tracking.

1.2 Core Architectural Decisions

Three patterns govern this design end to end:

- **CQRS (Command Query Responsibility Segregation)** — commands (writes) and queries (reads) are handled by physically separate models and, in this design, separate databases.
- **Event Sourcing** — instead of persisting only the latest state, every state change is appended as an immutable event to an Event Store, which becomes the single source of truth.
- **Eventual Consistency** — read models (Materialized Views) are rebuilt asynchronously from events published on the Eventbus, so reads may lag writes by a small, bounded interval.

> An additional pattern, the **Outbox Pattern**, is layered on top to solve the dual-write problem that occurs whenever a service must atomically persist state and publish an event.

2. Reference Architecture Diagram

The diagram below shows the complete request path: client traffic enters through the API Gateway, is routed to the relevant microservice's Command or Query side, and state changes propagate to every interested service's Read Database through the Kafka Eventbus.

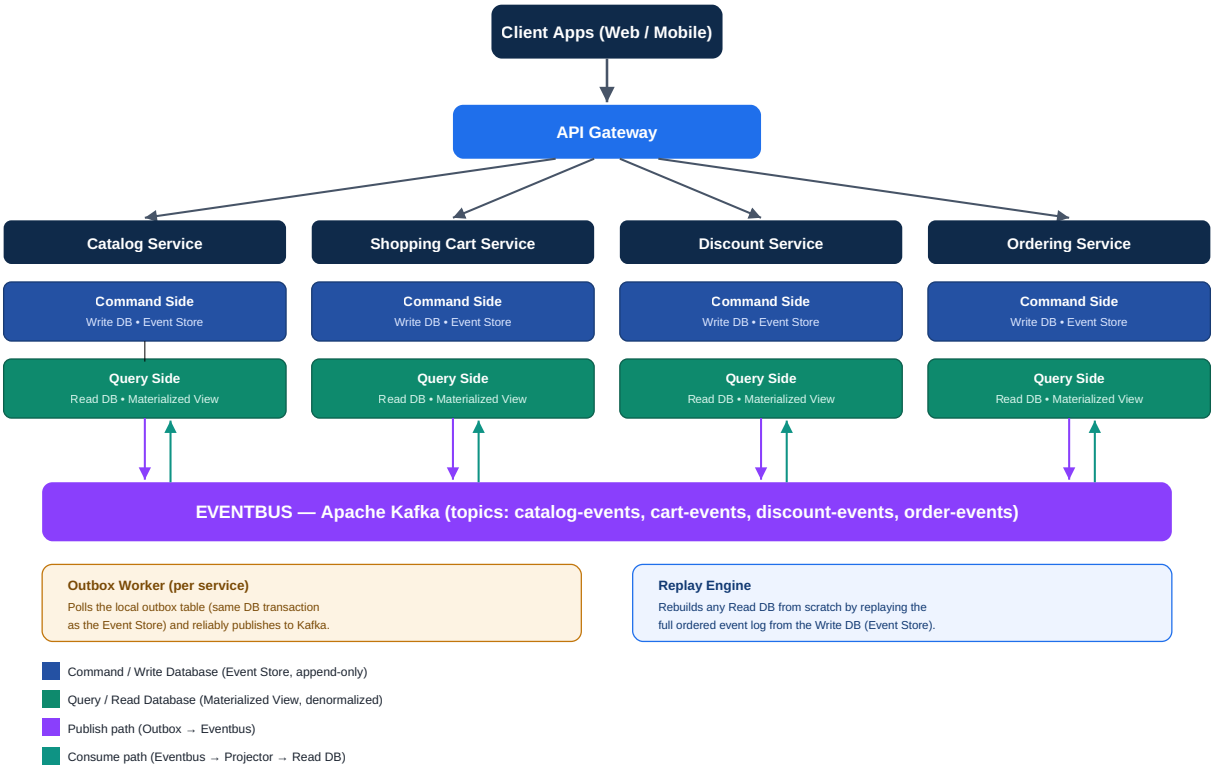


Figure 1 — Full CQRS + Event Sourcing reference architecture across the four microservices.

3. Microservices Characteristics

Characteristic	How It Is Applied
Componentization via Services	Each of the four services is an independently deployable Spring Boot application with its own codebase and database(s).
Organized Around Business Capabilities	Catalog, Cart, Discount and Ordering map 1:1 to distinct business domains.
Decentralized Data Management	Each service owns a separate Write DB (Event Store) and Read DB (Materialized View); no cross-service table sharing.
Smart Endpoints, Dumb Pipes	REST handles synchronous commands/queries; Kafka carries asynchronous domain events. No business logic lives in the broker.
Infrastructure Automation	Docker images per service; Kubernetes Deployments, Services, and StatefulSets for orchestration.
Design for Failure	Outbox pattern removes dual-write risk; consumers are idempotent; retries and dead-letter handling on the worker.
Evolutionary Design	Read models can be changed or added at any time and rebuilt via the Replay Engine without touching the Write DB.

4. Microservices Design Principles (CQRS & Event Sourcing)

4.1 Command-Query Separation

Every service exposes two distinct API surfaces:

- `/api/commands/...` — mutations, always routed to the Write DB.
- `/api/queries/...` — reads, always routed to the Read DB (Materialized View).

4.2 Event Sourcing Fundamentals

Instead of storing only the current state of an aggregate, every state-changing action is appended as an immutable event to an **Event Store**. The current state of any aggregate can always be recomputed by replaying its events in order. This gives us:

- A complete, auditable history of every change.
- The ability to rebuild any Read Database at any time (Replay).
- Natural support for temporal queries ("what was the state at version 5?").

4.3 Materialized Views

The Read Database contains denormalized, query-optimized tables — Materialized Views — built specifically to avoid joins and aggregation at query time. A single `SELECT ... WHERE id = ?` replaces what would otherwise require multiple joined tables.

4.4 Eventual Consistency

Because the Read DB is updated asynchronously by consuming events from Kafka, there is a small, bounded delay between a write being committed and the corresponding read model reflecting it. Clients should treat query results as "eventually" — not "immediately" — consistent.

4.5 Other Principles Carried Over

- **Single Responsibility** — one clear purpose per service.
- **Bounded Context** — independent domain models per service.
- **API First** — REST contracts defined upfront (OpenAPI/Swagger).
- **Statelessness** — services hold no in-memory session state; all state lives in a database.
- **Configuration Externalization** — environment-specific settings supplied via environment variables / ConfigMaps / Secrets.
- **Observability** — structured logs, health probes, and metrics on every service.

5. End-to-End Flow & Sequence Diagram

The scenario below traces a user placing an order: the command is durably recorded, published through the Outbox, and consumed by every interested service to update its own Read Database.

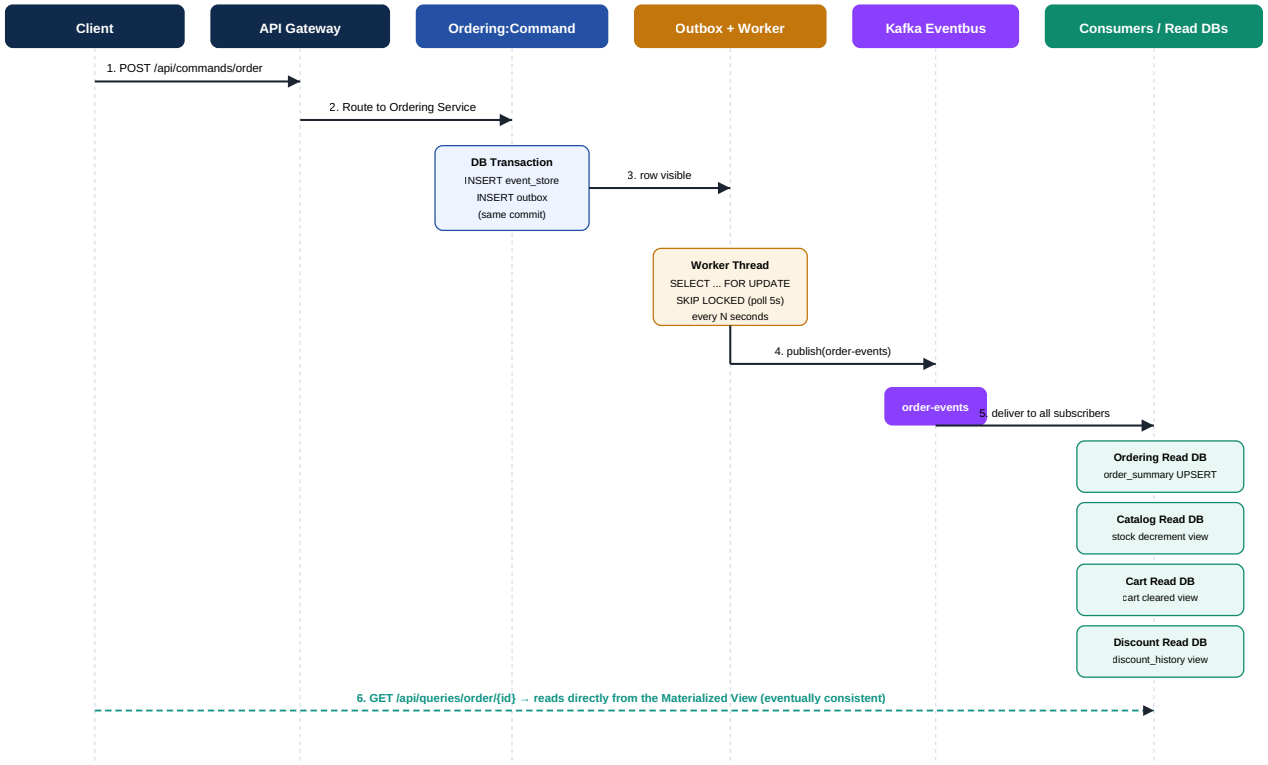


Figure 2 — Sequence flow for placing an order, from command intake to eventual consistency across services.

5.1 Narrative Walkthrough

1. The client submits `POST /api/orders`, entering through the API Gateway.
2. The Ordering Service validates the request, appends an `OrderPlaced` event to its Event Store, and writes a matching row to its local Outbox table — **in the same database transaction**.
3. An independent Worker Thread polls the Outbox table, publishes the event to the `order-events` Kafka topic, then marks/removes the outbox row.
4. Every interested consumer (Ordering, Catalog, Cart, Discount) receives the event and updates its own Materialized View: clearing the cart, decrementing stock, recording discount usage, and setting order status to `PLACED`.
5. The client can poll `GET /api/queries/order/{id}`, which reads straight from the Read DB — fast, join-free, but eventually consistent.

6. Technology Stack

Technology	Purpose
Java 17	Language
Spring Boot 3.x	Framework (Web, Data JPA, Kafka, Feign, Scheduling)
Spring Cloud Gateway	API Gateway — single entry point, routing, cross-cutting concerns
PostgreSQL	Write DB (Event Store + Outbox) and Read DB (Materialized Views) per service
Apache Kafka	Eventbus for asynchronous domain event propagation
Docker	Containerization of every service and dependency
Kubernetes	Orchestration — Deployments, Services, StatefulSets, ConfigMaps, Secrets
Maven	Build tool
Testcontainers	Integration testing against real PostgreSQL and Kafka containers
Awaitility	Asserting eventual consistency in tests

7. Read Database Replica Creation (Projection Pipeline)

The Read Database is **not** a physical database replica created through native replication (such as PostgreSQL streaming replication). It is a **logical replica** — a separate database rebuilt incrementally by consuming domain events. The table below shows the exact pipeline.

1. Write DB (Event Store)	A command executes; an `INSERT` appends a new row to `event_store`. Never an `UPDATE`.
2. Outbox & Worker	The same transaction records the event in the local `outbox` table; a worker thread later publishes it to Kafka.
3. Kafka (Eventbus)	The event is durably stored on the `order-events` topic.
4. Projector (Consumer)	A `@KafkaListener` consumes the event and projects it into the Read DB.
5. Read DB (Materialized View)	Denormalized rows are upserted — this is the "replica."

> Because it is derived, the Read DB can always be regenerated: replay every event from the Write DB, in version order, through the same projection logic, and the Materialized View is rebuilt from scratch — including after a schema change.

7.1 Projector Implementation

```
@Component
public class OrderEventProjector {   // Builds the logical "replica"

    @Autowired
    private OrderSummaryRepository readRepository;  // Points to READ_DB

    @KafkaListener(topics = "order-events", groupId = "projector-group")
    public void projectOrderPlaced(OrderPlacedEvent event) {
        OrderSummary summary = readRepository.findById(event.getOrderId())
                .orElse(new OrderSummary());

        summary.setOrderId(event.getOrderId());
        summary.setCustomerName(event.getCustomer());
        summary.setTotalAmount(event.getTotal());
        summary.setStatus("PLACED");
        summary.setItemCount(event.getItems().size());

        readRepository.save(summary);  // UPSERT into the Read DB
    }
}
```

7.2 Rebuilding After a Schema Change

```
@PostMapping("/api/replay/all")
public ResponseEntity<String> replayAll() {
    List<EventStore> allEvents = eventStoreRepo.findAllByOrderBySequenceIdAsc();
    materializedViewRepo.deleteAll();

    for (EventStore record : allEvents) {
        if ("OrderPlaced".equals(record.getEventType())) {
            OrderPlacedEvent event = fromJson(record.getPayload(), OrderPlacedEvent.class);
            OrderSummary summary = new OrderSummary();
            summary.setOrderId(event.getAggregateId());
            summary.setCustomerName(event.getCustomer());
            summary.setTotalAmount(event.getTotal());
            summary.setStatus("PLACED");
            materializedViewRepo.save(summary);
        }
    }
    return ResponseEntity.ok("Replayed " + allEvents.size() + " events.");
}
```

8. The Outbox Pattern & Worker Thread Management

8.1 The Dual-Write Problem

A command handler that both (a) writes to the database and (b) publishes to Kafka inside one method faces the dual-write problem: the two operations cannot be made atomic across two different systems. If the DB commit succeeds but the Kafka publish fails (or vice-versa), the Write DB and the Eventbus fall out of sync.

8.2 The Solution: Local Transaction + Outbox Table

The event is written to a local `outbox` table in the **same database transaction** as the domain write. A separate, independent worker thread later drains the outbox and publishes to Kafka — decoupling the DB commit from the network call to the broker.

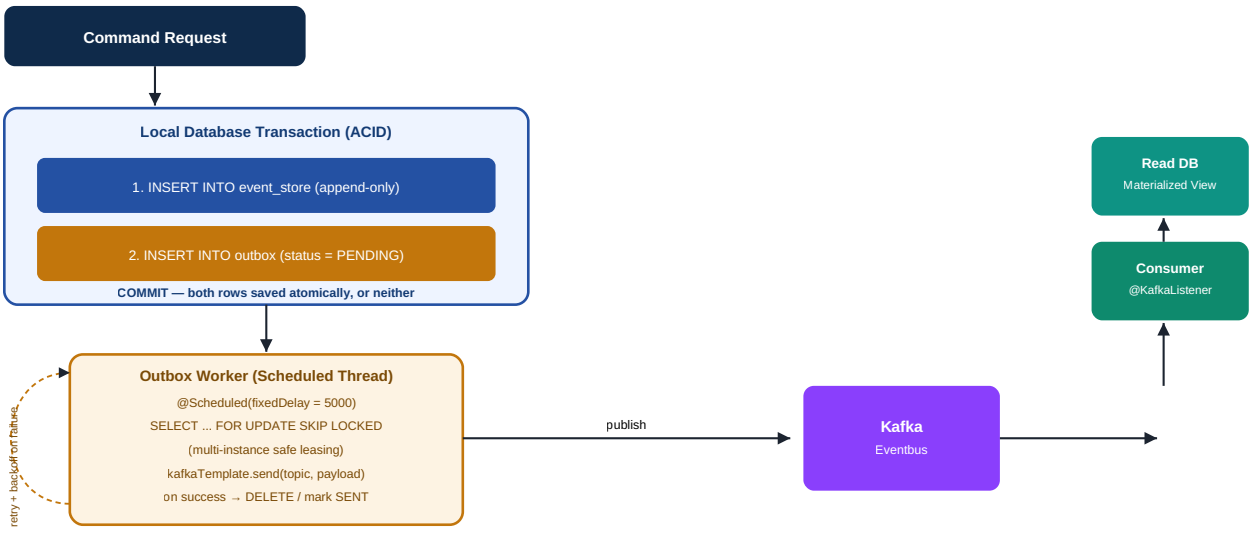


Figure 3 — Outbox pattern: local transaction guarantees atomicity; worker thread guarantees reliable, at-least-once delivery to Kafka.

8.3 Outbox Table Schema

```
CREATE TABLE outbox (
    id BIGSERIAL PRIMARY KEY,
    aggregate_id VARCHAR(50) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMP NOT NULL,
    status VARCHAR(20) DEFAULT 'PENDING',   -- PENDING, SENT, FAILED
    retry_count INT DEFAULT 0,
    locked_until TIMESTAMP NULL,            -- worker leasing
    version INT DEFAULT 0
);
CREATE INDEX idx_outbox_status_created ON outbox(status, created_at);
```

8.4 Command Handler — Single Local Transaction

```
@Service
@Transactional  // Covers BOTH the event_store INSERT and the outbox INSERT
public class OrderCommandService {

    @Autowired private EventStoreRepository eventStoreRepo;
    @Autowired private OutboxRepository outboxRepo;

    public void placeOrder(PlaceOrderCommand cmd) {
        EventStore record = new EventStore();
        record.setAggregateId(cmd.getOrderId());
        record.setEventType("OrderPlaced");
        record.setPayload(toJson(cmd));
        eventStoreRepo.save(record);          // INSERT (Event Store)

        Outbox outbox = new Outbox();
        outbox.setAggregateId(cmd.getOrderId());
        outbox.setEventType("OrderPlaced");
        outbox.setPayload(toJson(cmd));
        outbox.setCreatedAt(LocalDateTime.now());
        outbox.setStatus("PENDING");
        outboxRepo.save(outbox);              // INSERT (Outbox)

        // Kafka is NOT called here — transaction commits atomically.
    }
}
```

8.5 Worker Thread — Polling, Locking, Publishing

```
@Component
public class OutboxWorker {

    @Autowired private OutboxRepository outboxRepo;
    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;

    @Scheduled(fixedDelay = 5000)
    @Transactional(propagation = Propagation.REQUIRES_NEW)  // separate transaction
    public void processOutbox() {
        List<Outbox> pending = outboxRepo.findTop10ByStatusOrderByCreatedAtAsc("PENDING");

        for (Outbox record : pending) {
            try {
                kafkaTemplate.send("order-events", record.getAggregateId(),
record.getPayload())
                             .get(5, TimeUnit.SECONDS);
                outboxRepo.delete(record);           // success: remove / mark SENT

            } catch (Exception e) {
                record.setRetryCount(record.getRetryCount() + 1);
                record.setStatus(record.getRetryCount() > 5 ? "FAILED" : "PENDING");
                record.setLockedUntil(LocalDateTime.now().plusMinutes(1)); // backoff
                outboxRepo.save(record);
            }
        }
    }
}
```

8.6 Multi-Instance Safe Locking

When the service is scaled to multiple pods, each pod runs its own worker. To avoid two workers publishing the same event twice, rows are leased using `SELECT ... FOR UPDATE SKIP LOCKED`:

```
@Query("SELECT o FROM Outbox o WHERE o.status = 'PENDING' " +
       "AND (o.lockedUntil IS NULL OR o.lockedUntil < CURRENT_TIMESTAMP) " +
       "ORDER BY o.createdAt ASC")
List<Outbox> findAvailableRecords(Pageable pageable);
```

> Even if Kafka is completely unavailable, commands still succeed — events remain safely queued in the outbox table until the worker can deliver them. This removes Kafka availability from the critical path of the write.

9. Core Code Implementation

9.1 Event Store Entity (Write DB)

```
@Entity
@Table(name = "event_store")
public class EventStore {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long sequenceId;
    private String aggregateId;
    private String eventType;
    private String payload;       // JSON
    private LocalDateTime occurredOn;
    private int version;          // sequential per aggregate
}

public interface EventStoreRepository extends JpaRepository<EventStore, Long> {
    List<EventStore> findByAggregateIdOrderByVersionAsc(String aggregateId);
    List<EventStore> findAllByOrderBySequenceIdAsc();
}
```

9.2 Materialized View Entity (Read DB)

```
@Entity
@Table(name = "order_summary")
public class OrderSummary {
    @Id private String orderId;
    private String customerName;
    private BigDecimal totalAmount;
    private int itemCount;
    private String status;         // PLACED, SHIPPED, DELIVERED
    private LocalDateTime lastUpdated;
}

public interface OrderSummaryRepository extends JpaRepository<OrderSummary, String> {
    // No custom queries needed — data is already denormalized
}
```

9.3 Query Controller (Read Side)

```
@RestController
@RequestMapping("/api/queries/order")
public class OrderQueryController {
    @Autowired private OrderSummaryRepository materializedViewRepo;

    @GetMapping("/{orderId}")
    public ResponseEntity<OrderSummary> getOrder(@PathVariable String orderId) {
        return materializedViewRepo.findById(orderId)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

9.4 API Gateway Routing

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-commands
          uri: lb://ORDERING-SERVICE
          predicates: [ Path=/api/commands/order/** ]
        - id: order-queries
          uri: lb://ORDERING-SERVICE
          predicates: [ Path=/api/queries/order/** ]
        - id: catalog-commands
          uri: lb://CATALOG-SERVICE
          predicates: [ Path=/api/commands/product/** ]
        - id: catalog-queries
          uri: lb://CATALOG-SERVICE
          predicates: [ Path=/api/queries/product/** ]
```

10. Deployment — Docker & Kubernetes

10.1 Project Structure

```
microservices-cqrs-es/
├── api-gateway/
├── catalog-service/         (write-db + read-db + outbox worker)
├── shoppingcart-service/
├── discount-service/
├── ordering-service/
├── docker-compose.yml
└── k8s/
    ├── gateway-deploy.yaml, gateway-svc.yaml
    ├── ordering-write-db.yaml, ordering-read-db.yaml
    ├── catalog-write-db.yaml, catalog-read-db.yaml
    ├── kafka.yaml, zookeeper.yaml
    └── secrets.yaml
```

10.2 Docker Compose (Local Development)

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment: { ZOOKEEPER_CLIENT_PORT: 2181 }

  kafka:                      # Eventbus
    image: confluentinc/cp-kafka:latest
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  ordering-write-db:          # Event Store + Outbox
    image: postgres:15
    environment: { POSTGRES_DB: write_db, POSTGRES_USER: admin, POSTGRES_PASSWORD: admin }
    ports: ["5432:5432"]

  ordering-read-db:           # Materialized Views
    image: postgres:15
    environment: { POSTGRES_DB: read_db, POSTGRES_USER: admin, POSTGRES_PASSWORD: admin }
    ports: ["5433:5432"]

  ordering-service:
    build: ./ordering-service
    depends_on: [ordering-write-db, ordering-read-db, kafka]
    ports: ["8080:8080"]
    environment:
      WRITE_DB_URL: jdbc:postgresql://ordering-write-db:5432/write_db
      READ_DB_URL:  jdbc:postgresql://ordering-read-db:5432/read_db
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092

  api-gateway:
    build: ./api-gateway
    ports: ["8000:8080"]
```

10.3 Kubernetes Manifest — Ordering Service

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ordering-service
spec:
  replicas: 3
  selector: { matchLabels: { app: ordering-service } }
  template:
    metadata: { labels: { app: ordering-service } }
    spec:
      containers:
      - name: ordering-service
        image: myrepo/ordering-service:1.0
        ports: [{ containerPort: 8080 }]
        env:
        - name: WRITE_DB_URL
          value: jdbc:postgresql://postgres-ordering-write:5432/write_db
        - name: READ_DB_URL
          value: jdbc:postgresql://postgres-ordering-read:5432/read_db
        - name: SPRING_KAFKA_BOOTSTRAP_SERVERS
          value: kafka-service:9092
---
apiVersion: v1
kind: Service
metadata: { name: ordering-service }
spec:
  selector: { app: ordering-service }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
```

10.4 Deployment Topology Diagram

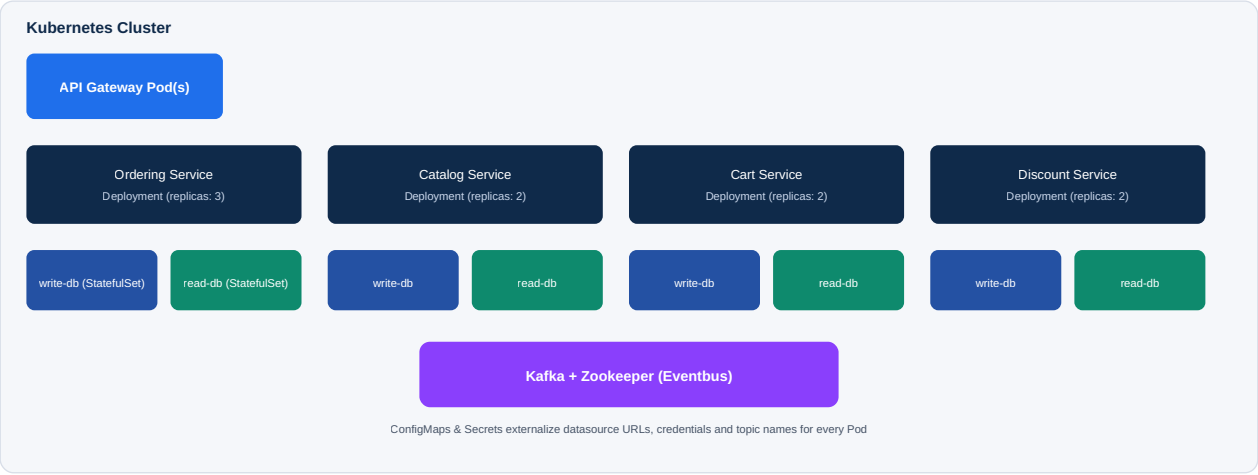


Figure 4 — Kubernetes deployment topology: each service owns an isolated Write DB / Read DB StatefulSet pair, all connected via a shared Kafka eventbus.

10.5 Operational Commands

```
# Local development
docker build -t myrepo/order-service:1.0 ./ordering-service
docker push myrepo/order-service:1.0
kubectl apply -f k8s/

# Scaling
kubectl scale deployment ordering-service --replicas=3

# Resilience test
kubectl delete pod <ordering-service-pod-name>

# Minikube
minikube start
kubectl get pods
kubectl expose deployment order-service --type=NodePort --port=8080
```

11. Microservices Testing Strategy

11.1 Unit Tests

Verify command handlers append to the Event Store and Outbox exactly once, and never issue an `UPDATE` against the append-only event log.

```
@ExtendWith(MockitoExtension.class)
class OrderCommandServiceTest {
    @Mock private EventStoreRepository eventStoreRepo;
    @Mock private OutboxRepository outboxRepo;
    @InjectMocks private OrderCommandService service;

    @Test
    void placeOrder_shouldAppendEventAndOutboxRow() {
        service.placeOrder(new PlaceOrderCommand("order-123", "John", BigDecimal.TEN));
        verify(eventStoreRepo).save(any(EventStore.class));
        verify(outboxRepo).save(any(Outbox.class));
    }
}
```

11.2 Integration Tests (Testcontainers)

```
@SpringBootTest
@Testcontainers
class OrderIntegrationTest {
    @Container static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:15");
    @Container static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:latest"));

    @Autowired private TestRestTemplate restTemplate;

    @Test
    void shouldEventuallyUpdateMaterializedView() {
        restTemplate.postForEntity("/api/commands/order", request, Void.class);

        await().atMost(5, TimeUnit.SECONDS).until(() -> {
            OrderSummary s = restTemplate.getForObject(
                "/api/queries/order/order-123", OrderSummary.class);
            return s != null && "PLACED".equals(s.getStatus());
        });
    }
}
```

11.3 Replay & Resilience Tests

- Delete a row from the Materialized View, call the replay endpoint, and confirm it is fully restored from the Event Store.
- Stop the Kafka container mid-test and confirm commands still succeed (events remain queued in the outbox).
- Kill a service pod and confirm Kubernetes reschedules it with no event loss.

11.4 Contract Testing

Spring Cloud Contract defines the REST contract for each service's command/query API so that consumer services can verify compatibility before deployment.

12. Containers & Orchestration

12.1 Containers (Docker)

- Each microservice, its write database, read database, and the Kafka broker are packaged as independent, portable images.
- Consistent runtime behaviour is guaranteed across local, staging, and production environments.

12.2 Orchestration (Kubernetes)

- Deployments manage rolling updates and self-healing for each service.
- Services provide stable internal DNS names (e.g. `ordering-service`, `kafka-service`).
- StatefulSets with PersistentVolumeClaims protect the Write DB (Event Store) and Read DB from data loss.
- ConfigMaps and Secrets externalize connection strings, credentials, and topic names.
- Liveness and readiness probes (`/actuator/health`) ensure traffic is only routed to healthy pods.

13. Summary Mapping & Benefits

Command	`POST /api/commands/order`
Write Database / Event Store	WRITE `event_store` table — append-only, source of truth
Outbox	Same local transaction as the Event Store write — guarantees atomicity
Eventbus	BUS Apache Kafka topic `order-events`
Worker Thread	Scheduled poller with `SELECT ... FOR UPDATE SKIP LOCKED`, retries with backoff
Consume Event	`@KafkaListener` in every interested service
Read Database / Materialized View	READ `order_summary` table — denormalized, join-free
Eventual Consistency	Bounded asynchronous delay between write commit and read-model update
Replay Events	`POST /api/replay/all` — rebuilds any Read DB from the Event Store

13.1 Benefits Achieved

Requirement	How It Is Delivered
Avoid complex queries / joins	Read DB stores fully pre-joined, denormalized rows per query need.
Independent read/write scaling	Write DB and Read DB are physically separate databases (can even use different engines, e.g. PostgreSQL for writes, MongoDB for reads).
No dual-write inconsistency	Outbox pattern makes the DB write and the "intent to publish" atomic in one local transaction.
No event loss during broker outages	Events remain safely queued in the outbox table until the worker can deliver them.
Full auditability & recoverability	Event Store retains every state change; Read DB can always be rebuilt via Replay.
Horizontal scalability	`SKIP LOCKED` outbox leasing allows multiple worker instances to run safely in parallel.

This document consolidates the complete architecture design — introduction, characteristics, design principles, end-to-end flow, technology stack, deployment, testing, containers/orchestration, read-model replica creation, and the Outbox pattern with worker-thread management — into a single reference for implementation teams.